How-To Library

Informatica

Incremental update strategies for relational tables in Data Engineering

© Copyright Informatica LLC 2021. Informatica and the Informatica logo are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https://www.informatica.com/trademarks.html.

Abstract

Read this article to learn how to use the Update Strategy transformation to update relational database sources to support incremental loads and ensure that targets are in sync with source systems. This article describes how to update relational databases and offers an example use case of this implementation.

Supported Versions

- Data Engineering 10.4.0 and later

Table of Contents

Overview. 2
Using the Update Strategy transformation. 2
Rules and guidelines for using the Update Strategy transformation with relational tables. 3
Example. 4
Configure the first mapping. 5
Configure the second mapping. 8

Overview

You can build data lakes on Hadoop clusters to perform near real-time analysis of your data. To build a data lake, perform an initial load, and then frequently make incremental updates. To make incremental updates to JDBC-compliant relational targets in the Hadoop environment, use the Update Strategy transformation.

Using the Update Strategy transformation

You can use an Update Strategy transformation to update relational tables after the initial load.

Perform the following steps to use an Update Strategy transformation:

1. Add an Update Strategy transformation to the mapping.

Add an Update Strategy transformation the mapping and link the incoming ports.

2. Define an update strategy expression.

Use DECODE or IIF functions in the expression to set rules for updating rows. The DECODE function searches a port for specified values and returns a result value if the search value is found or a default result if the search value is not found. The IIF function returns one of two values, depending on a specified condition. You can nest IIF functions to test multiple conditions.

Use the following constants in the update strategy expression to flag rows for insert, update, delete, or reject:

Operation	Constant	Numeric Value
Insert	DD_INSERT	0
Update	DD_UPDATE	1

Operation	Constant	Numeric Value
Delete	DD_DELETE	2
Reject	DD_REJECT	3

For example, the following IIF statement flags a row for reject if the entry date is after the apply date. Otherwise, the statement flags the row for update: `IIF( ( ENTRY_DATE > APPLY_DATE), DD_REJECT, DD_UPDATE)`

3. Configure advanced properties for the target data object.

After you configure the Update Strategy transformation, you can configure the following advanced properties for the target data object in the mapping:

Insert

Inserts all rows flagged for insert into a target. Default is enabled.

Delete

Deletes all rows flagged for delete from a target. Default is enabled.

Update Strategy

Update strategy for existing rows. Select one of the following strategies:

- **Update as Update.** Updates all rows flagged for update. This is the default value.
- **Update as Insert.** Inserts all rows flagged for update.
- **Update else Insert.** Updates all rows flagged for update if they exist in the target and then inserts any remaining rows marked for insert.

Truncate Table

Truncates the target before loading data. Default is disabled.

Rules and guidelines for using the Update Strategy transformation with relational tables

When you use the Update Strategy transformation in a mapping that writes to a relational target, you must run the mapping on the Spark engine. Some processing rules for relational data sources differ based on the run-time engine used to run the mapping.

Using Hierarchical Data in Update Strategy Expressions

You can configure the expression to use the primitive types in a complex data field.

For example, you have a nested array of struct data types defined by the following statement:

```
addresses array<struct<city:varchar(15), apartment:smallint, pincode:int>>
```

You can use the city column in the update strategy expression since it is a primitive type using the following expression:

```
IIF( (addresses[0].city = 'NEW YORK'), DD_INSERT, DD_UPDATE)
```

Mapping Validation

Mapping validation fails in the following situations.

- The Update Strategy transformation is connected to more than one target.

- The Update Strategy transformation is not located immediately before the target.
- The Update Strategy transformation target is an external ACID table.
- The target does not contain a connected primary key.
- The property to enable truncation of the target table at run time is selected.
- One of the following target strategies for the target table at run time is selected:
  - Create or replace the target table
  - ApplyNewColumns
  - ApplyNewSchema
  - Fail

Example

You are a data engineer for a bank, tasked with regularly updating a database of customer accounts, that includes customer names, mailing addresses, and account numbers.

You have one table in your target database that acts as your data lake, *CustomerInfo*, and one table that contains new and recently updated records, *UpdatedInfo*. The updated records table includes information for new customers and for existing customers whose address has changed.

Both tables contain the following columns: AccountNumber, CustomerName, MailingAddress. *CustomerInfo* also includes EffectiveDate and EndDate columns. The EffectiveDate and EndDate columns are used to track changes to records. When a new record is inserted, the EndDate is NULL. When the record is updated, the EndDate is added for the existing record and a new record is inserted.

For example, when a customer join the bank, an initial record of their information is stored in *CustomerInfo*:

AccountNumber	CustomerName	MailingAddress	EffectiveDate	EndDate
10003	Amit Walia	2100 Seaport Blvd	01-01-2020 00:00:00	

When the customer updates their mailing address, a record is added to *UpdatedInfo* with the customer's existing account number, their name, and their new mailing address. Similarly, when a new customer joins, their information is first added to *UpdatedInfo*.

AccountNumber	CustomerName	MailingAddress
10003	Amit Walia	120 Bremner Blvd
10005	Jeff Bezos	2111 7th Ave

When *UpdatedInfo* is used to update the data lake table, the current date is used as the end date of the existing record and a new row is inserted.

AccountNumber	CustomerName	MailingAddress	EffectiveDate	EndDate
10003	Amit Walia	2100 Seaport Blvd	01-01-2020 00:00:00	01-08-2020 00:00:00
10003	Amit Walia	120 Bremner Blvd	01-08-2020 00:00:00	
10005	Jeff Bezos	2111 7th Ave	01-08-2020 00:00:00	

To update the data lake as a slowly-changing dimension (SCD) type 2 table, you need mappings to perform the following tasks:

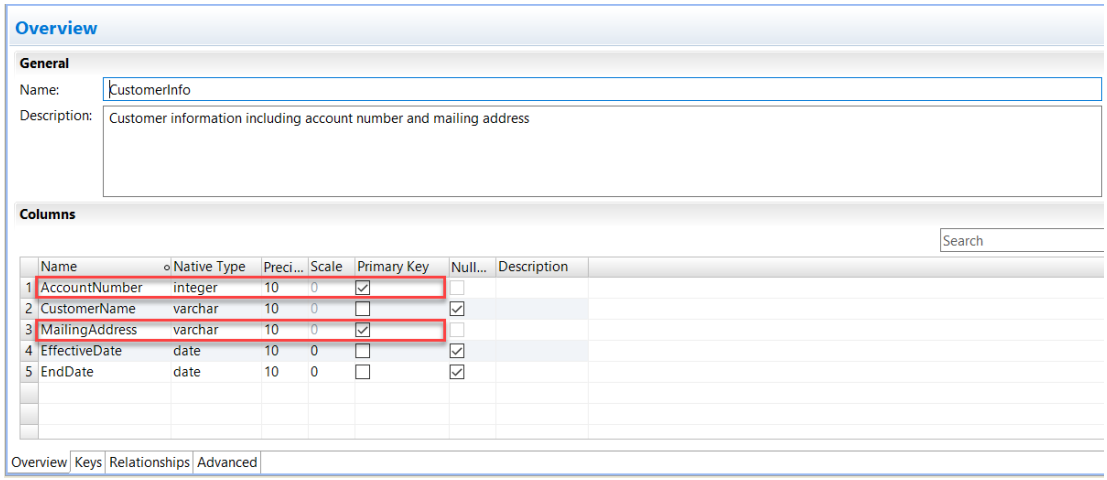
- When a new customer joins the bank, create a record with their information.
- When an existing customer updates their mailing address, update the end date of their existing record and insert a new record with their updated information.

To implement the SCD type 2 table, create a workflow with two mappings. The first mapping updates the existing records for existing customers. The second mapping inserts new records for existing customers and records for new customers.

Configure the first mapping

Configure a mapping to add end dates to existing records that are updated.

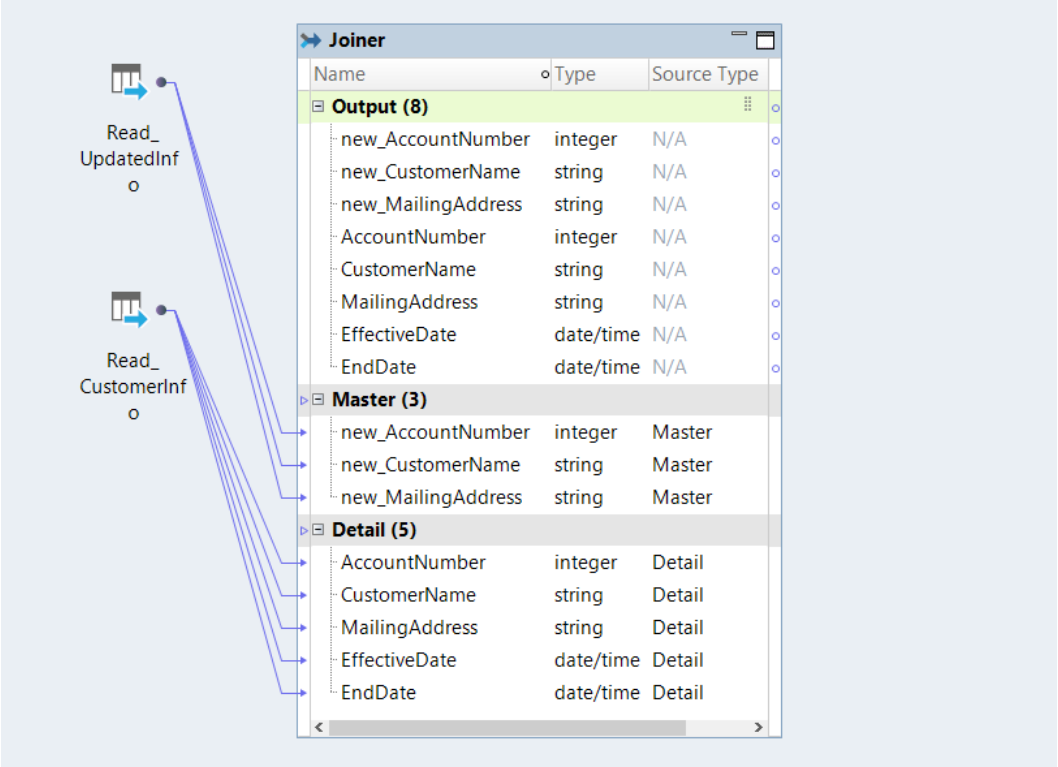
In properties for the *CustomerInfo* data object, ensure that AccountNumber and MailingAddress are configured as the composite primary key.



1. Create a mapping that reads from *CustomerInfo* and *UpdatedInfo*.
2. Add a Joiner transformation to the mapping to join the two sources. In the join condition, select only rows that need to be updated.
   a. Add the ports from *UpdatedInfo* as the master ports. In the **Ports** properties, append "new_" to the port names.
   b. Add the ports from *CustomerInfo* as the detail ports.
   c. In the **Join** properties, set the **Join Type** to *Normal*.

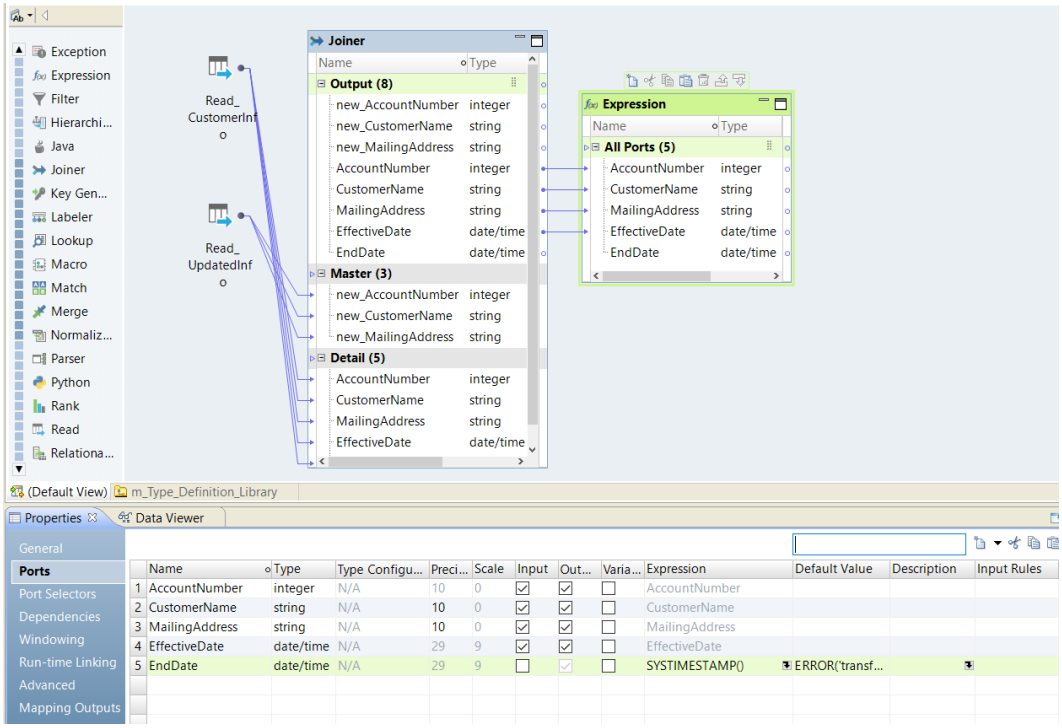
d. Define an advanced condition where `AccountNumber = new_AccountNumber AND ISNULL(EndDate)`.

The following image shows the Joiner transformation in the mapping:



3. Add an Expression transformation to add end dates to the records.
   a. Connect the following Joiner transformation output ports to the Expression transformation: AccountNumber, CustomerName, MailingAddress, EffectiveDate.
   b. In the **Ports** properties, create a port with the name *EndDate*. Set the type to **date/time** and configure the port for output only. Set the expression for the port to `SYSTIMESTAMP()`.

The following image shows the Expression transformation configuration:



4. Add an Update Strategy transformation to flag the records for update.

   In the Update Strategy properties, set the following update strategy expression:

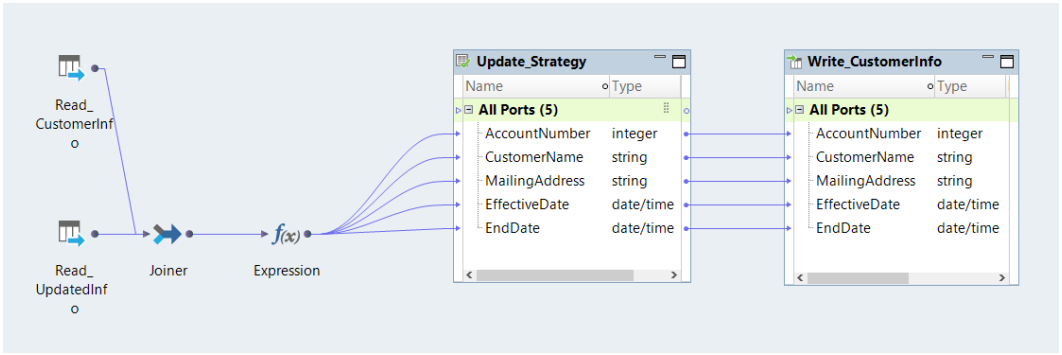
   ```
   DD_UPDATE
   ```

5. Add a Write transformation to the mapping.

   a. Create the transformation using the *CustomerInfo* data object.

   b. In the Advanced properties, ensure that **Update Strategy** is set to *Update as Update*.

   The following image shows the Write transformation in the mapping:



After this mapping runs, the *CustomerInfo* table contains the following information:

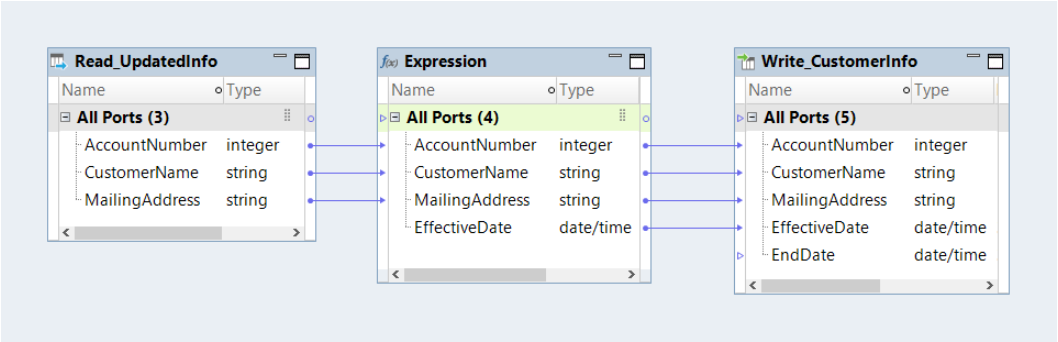
AccountNumber	CustomerName	MailingAddress	EffectiveDate	EndDate
10003	Amit Walia	2100 Seaport Blvd	01-01-2020 00:00:00	01-08-2020 00:00:00

Configure the second mapping

Configure a mapping to insert updated records for existing customers and new records for new customers.

1. Create a mapping that reads from *UpdatedInfo*.
2. Add an Expression transformation to add the effective dates to all the records.
   a. Create a port with the name *EffectiveDate*.
   b. Set the port type to **date/time** and configure the port for output only.
   c. Set the expression for the port to `SYSTIMESTAMP()`.
3. Add a Write transformation to the mapping.
   a. Create the transformation using the *CustomerInfo* data object.
   b. In the Advanced properties, ensure that **Insert** is enabled.

   The following image shows the final mapping configuration:



After this mapping runs, the *CustomerInfo* table contains the following information:

AccountNumber	CustomerName	MailingAddress	EffectiveDate	EndDate
10003	Amit Walia	2100 Seaport Blvd	01-01-2020 00:00:00	01-08-2020 00:00:00
10003	Amit Walia	120 Bremner Blvd	01-08-2020 00:00:00	
10005	Jeff Bezos	2111 7th Ave	01-08-2020 00:00:00	

Author

Angela Rae

Acknowledgements

The author would like to acknowledge Peeyush Agrawal, Ninju Bijily, and Mir Mohamed Salman for their assistance with this article.